

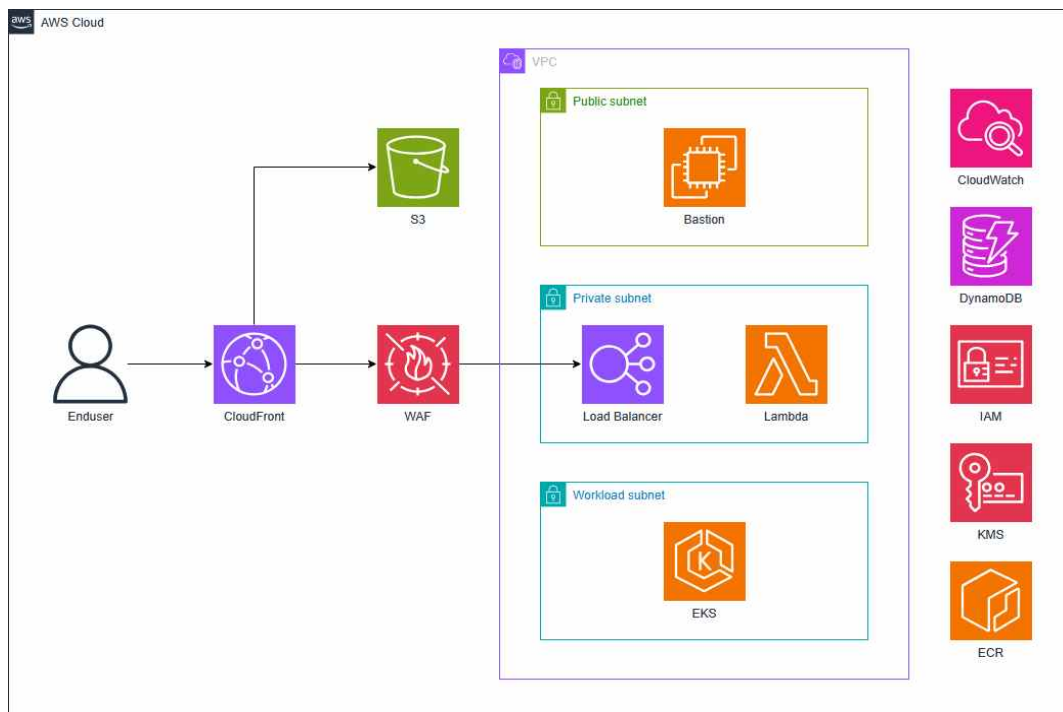
2026년도 전국기능경기대회

직 종 명	클라우드 컴퓨팅	과 제 명	Solution Architecture	과제번호	제 1과제
경기시간	4시간	비 번 호		심사위원 확 인	(인)

1. 요구사항

웹 서비스를 제공하기 위한 REST API를 배포하고 운영하고자 합니다. REST API는 추후 확장성, 안정성 등을 고려하여 MSA(MicroService Architecture) 패턴으로 설계되고 개발되었습니다. 또한 Zero Trust Network Access(ZTNA) 보안 모델을 기반으로, 모든 사용자 및 서비스 간 통신에 대해 지속적인 인증과 인가를 수행하도록 설계되었습니다. 이러한 보안 요구사항을 인프라 수준에 일관되게 적용하기 위해 컨테이너 기반 환경을 채택하고, 효율적인 서비스 운영 및 확장을 위해 Kubernetes를 오케스트레이션 도구로 사용합니다. 더불어 서비스 간 통신은 인증 및 암호화를 기본으로 하며, 최소 권한 원칙을 적용하여 접근을 제어합니다. 이와 함께 성능, 보안, 고가용성, 운영 효율성, 지속가능성 등 여러 요소를 고려하여 어플리케이션을 배포하고 인프라를 구축하세요.

다이어그램



Software Stack

AWS		CNCF/Tools	Language
- VPC	- CloudFront	- Prometheus	- Go
- EC2	- IAM	- Grafana	- Python
- ELB	- KMS	- FluentBit	
- EKS	- Lambda		
- ECR	- WAF		
- DynamoDB	- CloudWatch		
- S3			

2. 선수 유의사항

※ 다음 유의사항을 고려하여 요구사항을 완성하시오.

- 1) 문제에 제시된 괄호박스 < >는 변수를 뜻함으로 선수가 적절히 변경하여 사용해야 합니다.
- 2) 문제 풀이와 채점의 효율을 위해 Security Group의 80/443 Outbound는 Anyopen하여 사용할 수 있도록 합니다.
- 3) Bastion EC2는 채점 시 사용되기 때문에 종료되거나 연결 문제, 권한 문제 등으로 발생할 수 있는 불이익을 받지 않도록 주의하시기를 바랍니다.
- 4) 모든 리소스는 서울(ap-northeast-2) 리전에 구성합니다.
- 5) 제공자료는 수정 없이 사용합니다. 제공자료를 수정해서 사용하면 불이익을 받을 수 있습니다.
- 6) 문제에서 주어지지 않는 값들은 AWS Well-Architecture Framework 6 pillars를 기준으로 적절히 값을 설정해야 합니다.
- 7) 불필요한 리소스를 생성할 경우, 감점의 요인이 될 수 있습니다. (e.g. VPC 추가 생성)
- 8) 모든 리소스의 이름, 태그, 변수는 대소문자를 구분합니다.
- 9) 1페이지의 다이어그램은 구성을 추상적으로 표현한 그림으로 세부적인 구성은 아래 요구사항을 만족시킬 수 있도록 합니다. (ex. 서버넷이 2개 이상 존재할 수 있습니다.)

3. Application

Go에서 개발된 binary book이 배포됩니다. 자세한 사항은 Reference02를 참고하여 구성합니다.

4. VPC

VPC를 사용하여 Cloud Networking을 구성합니다. 클라우드 인프라에 대해 네트워크 레벨의 격리 및 분리가 가능하도록 VPC를 구성합니다. Workload Subnet에 Endpoint를 사용하여 AWS Service에 대해 통신이 가능하여야 하며, 연결되는 Route Table에 대해서는 라우팅 테이블에 어떠한 규칙도 포함되지 않아야 합니다. 자세한 사항은 Reference01을 참고하여 구성합니다.

5. Bastion

채점 진행 시 EKS Node Group 접근 및 채점 진행을 위해 Bastion 서버를 구성합니다. 해당 서버에 접근 불가할 시 채점이 불가하므로 반드시 SSH를 통한 접속과 권한 문제가 없도록 합니다. 서버가 재시작하여도 Bastion IP가 변동되지 않게 구성합니다. 외부에서 SSH 프로토콜만을 허용하도록 Security Group을 구성합니다. 또한 채점 진행 시 SSH Password 방식을 통해 Bastion에 접근하며, Password는 "Skill53##"으로 지정합니다. Bastion은 Public Subnet에 배포하며, Bastion은 awscli 입력 시 Admin Policy에 상응하는 권한을 가지고 있어야 합니다. 또한 채점 진행 시 sshpass를 사용하므로 유의하여 구성합니다.

- EC2 Instance Type: t3.medium
- Image: Amazon Linux 2023
- Package: awscli, jq, curl, ping, kubectl, eksctl
- Tag: Name=wsc-bastion

6. S3

정적 파일을 저장하기 위하여 S3 Bucket을 생성합니다. 해당 S3 Bucket에는 제공된 배포 파일을 /static에 업로드합니다. 또한 S3 Bucket 및 Object에 대해서 SSE-KMS 암호화를 사용하여 보안을 강화하도록 합니다.

- S3 Bucket Name: wsc-static-<ACCOUNT_ID>

7. Container Registry

Docker Image를 저장할 ECR을 구성합니다. ECR에 업로드된 이미지들은 KMS 암호화와 취약점 분석이 가능해야 하며, 어떠한 취약성도 존재해서는 안 됩니다. 또한 이미지 전송 시간 단축과 배포 효율성을 위해, Docker Image의 크기를 8MB 이하로 경량화하여 업로드합니다. 채점 진행 시 컨테이너에 curl 요청을 수행하므로, 미리 설치해두도록 합니다.

- ECR Name: wsc-repo
- Image Tag: v1.0.0

8. DynamoDB

애플리케이션의 데이터를 저장하기 위해 NoSQL 기반 데이터베이스인 DynamoDB를 구성합니다. 보안 강화를 위하여 Customer Managed Key로 Encryption을 하도록 합니다.

- Table Name: wsc-table
- Description of table

Key Name	Data Type	ETC
client_id	String	Partition Key
username	String	-
email	String	-
concert_name	String	-

9. Container Orchestration

제공된 Application을 Container 환경에 배포하기 위해 EKS를 사용합니다. EKS Cluster Control Plane에서 발생하는 모든 로그들을 CloudWatch Logs에서 확인할 수 있어야 하며, Secret Resource들은 반드시 KMS Encryption 되어야 합니다. 또한 Kubernetes API는 외부에서 접근 불가능해야 하며, Bastion Server에서만 접근할 수 있어야 합니다. 관리의 편의를 위해 모든 Node Group은 Managed NodeGroup으로 생성하며, 최소한의 고가용성을 고려해야 합니다. 호스트를 위한 EC2의 운영체제는 Amazon Linux 2023을 사용합니다. EKS는 Workload Subnet에서 운용 되어야 합니다.

9.1) Cluster

- EKS Cluster Name: wsc-eks-cluster
- EKS Cluster Version: 1.35
- EKS Encryption: KMS
- Public Access의 경우 비활성화하며, Private Access만 활성화합니다.

9.2) Node Group

Node의 hostname을 <INSTANCE_ID>.ap-northeast-2.compute.internal로 변경합니다. Kubernetes 내부에서 사용하는 Domain을 기존 *.cluster.local에서 *.wsc.local로 변경합니다. 또한 Bastion에서 SSH Passowrd 방식을 통해 NodeGroup에 접속이 가능해야하며, Password는 "Skill53##" 을 사용합니다. curl 및 ping 명령어가 작동하도록 설정합니다. Node Group의 Storage는 KMS를 통해 암호화 되어야 합니다.

App Managed NodeGroup

Application들은 반드시 Application NodeGroup에서 운용되어야 합니다. 이 외의 다른 Resource들이 존재해서는 안 됩니다. 또한 해당 NodeGroup의 Node는 { type: app }라는 Label을 가지고 있어야 합니다.

- EKS App Node Group Name: wsc-app-ng
- EKS App Node Instance Name Tag: wsc-app-node
- EKS App Node Instance Type: t3.medium

Addon Managed NodeGroup

Application을 제외한 AWS Load Balancer Controller와 같은 Addon들은 반드시 Addon NodeGroup에서 운용되어야 합니다. 또한 해당 NodeGroup의 Node는 { type: addon }라는 Label을 가지고 있어야 합니다.

- EKS Addon Node Group Name: wsc-addon-ng
- EKS Addon Node Instance Name Tag: wsc-addon-node
- EKS Addon Node Instance Type: t3.medium

Monitoring Managed NodeGroup

Prometheus와 Grafana는 반드시 Monitoring NodeGroup에서 운용되어야 합니다. 또한 해당 NodeGroup의 Node는 { type: monitoring }라는 Label을 가지고 있어야 합니다.

- EKS Monitoring Node Group Name: wsc-monitoring-ng
- EKS Monitoring Node Instance Name Tag: wsc-monitoring-node
- EKS Monitoring Node Instance Type: t3.medium

9.3) Kubernetes Resource

애플리케이션은 “wsc” 라는 Namespace를 사용하여 EKS Cluster에서 논리적으로 분리시켜야 하며, Kubernetes Deployment를 통해 관리되어야 합니다. 항상 고가용성을 유지할 수 있도록 구성합니다. Deployment의 Label은 { app: wsc-deploy }를 사용합니다.

- Deployment Name: wsc-deploy
- Container Name: wsc-cnt

9.4) Application Pod

App NodeGroup에 존재하는 Pod는 EC2 Instance의 IAM 권한을 사용할 수 없게 구성합니다. IAM 권한이 필요한 경우 ServiceAccount를 사용하도록 합니다.

9.5) Environment Configuration Management

환경변수를 Pod에 직접 명시하지 않도록 구성합니다. 환경변수는 Kubernetes ConfigMap을 통해 관리하며, Pod에서는 해당 값을 참조하여 사용하도록 한다. 이를 통해 애플리케이션 설정 값을 중앙에서 일관되게 관리하고, 환경 변수의 직접 하드코딩을 방지합니다.

- ConfigMap Name: wsc-config

9.6) Storage Management

데이터 영속성 보장을 위해 EBS CSI Driver 기반의 영구 저장소를 구성합니다. 모든 볼륨은 Customer Managed Key 로 Encryption 되어야 합니다. 볼륨은 고가용성을 위해 Pod가 실제 배치되는 가용 영역(AZ)에 맞춰 동적으로 프로비저닝 되어야 하며, 각 서비스는 지정된 네임스페이스 내 PVC를 통해 용량을 할당받아 Pod 재시작 시에도 데이터가 유실되지 않고 기존 경로에 정상적으로 마운트 되도록 설정합니다.

- StorageClass Name: wsc-sc
- Persistent Volume Claim Name: wsc-prometheus-pvc, wsc-grafana-pvc

10. CloudWatch Logs

Fluent Bit를 사용하여 Application 로그를 CloudWatch Logs에 기록되도록 구성합니다. Fluent Bit는 DaemonSet 방식을 사용하도록 하며, Pod에서 남기는 app 로그가 Cloudwatch Log에 저장되도록 구성합니다. 해당 서비스들은 “logging” 이라는 Namespace를 사용하여 EKS Cluster에서 논리적으로 분리시켜야 합니다. 또한 /health는 로그에 저장되지 않게 구성합니다. 최대 1분안에 CloudWatch Logs에 로그에 수집되어야 합니다. 보안 강화를 위하여 모든 CloudWatch Logs는 KMS로 Encryption을 하도록 합니다.

- DaemonSet Name: fluent-bit
- Log Group Name: /wsc/pod/log
- Log Stream Name: /wsc/app/log

11. Observability & Monitoring

Prometheus와 Grafana를 사용하여 Observability & Monitoring을 구성합니다. 해당 서비스들은 “monitoring” 이라는 Namespace를 사용하여 EKS Cluster에서 논리적으로 분리시켜야 합니다.

11-1) Observability

Prometheus는 node-exporter 및 kube-state-metrics를 통해 Node Group에 속한 모든 Node와 Pod의 CPU 및 Memory 사용량을 수집하여야 합니다.

11-2) Monitoring

Grafana는 Prometheus를 Source로 설정하여 Kubernetes의 정보를 불러오도록 구성합니다. datasource는 `http://prometheus-server.monitoring.svc.wsc.local/prometheus`을 사용하도록 합니다.

- Dashboard Name: `wsc-eks-dashboard`

- User Information:

User Name	Password
admin	Skill53##

- Cluster Summary

Panel Name	Type	Description
TOTAL_NODE_GROUP_COUNT	Stat	전체 Node Group 수
APP_POD_COUNT	Stat	Application Pod 수

- Resource Utilization

Panel Name	Type	Description
NODE_GROUP_CPU_USAGE	Time Series	Node Group 별 CPU 사용률
NODE_GROUP_MEMORY_USAGE	Time Series	Node Group 별 Memory 사용률
APP_POD_CPU_USAGE	Bar Gauge	Application Pod CPU 사용률 100분위로 표시
APP_POD_MEMORY_USAGE	Bar Gauge	Application Pod Memory 사용률 100분위로 표시

12. Elastic Load Balancer

12.1) App Load Balancer

L7에서 동작하는 Load Balancer를 구성합니다. 해당 Load Balancer는 Private Subnet에 운용되어야 하며, 외부에서 접근이 불가능 하여야 합니다. Application에 명시된 API를 제외한 접근은 404 "Contents Not Found"를 반환하도록 합니다. Security Group을 이용하여 CloudFront를 통해서만 접근할 수 있도록 설정합니다. 자세한 사항은 아래를 참고하여 구성합니다.

- Load Balancer Name: `wsc-app-lb`

- Listen Port:

Listen Port	Description
80	wsc Namespace에서 운영하는 웹 서비스

12.2) Addon Load Balancer

L7에서 동작하는 Load Balancer를 구성합니다. 해당 Load Balancer는 Public Subnet에 운용되어야 하며, 외부에서 접근이 가능 하여야 합니다. 자세한 사항은 아래를 참고하여 구성합니다.

- Load Balancer Name: `wsc-addon-lb`

- Listen Port:

Listen Port	Path	Description
80	/grafana	Grafana의 메인 웹 서비스
	/prometheus	Prometheus의 메인 웹 서비스

13. WAF

L7에서 동작하는 Firewall을 구성하여 웹 애플리케이션에 대한 보안을 강화합니다. CloudFront에 연결하여 요청을 검사하도록 구성합니다. POST Method 요청 시 Body에 admin 및 sysop 문자열이 대소문자 구분 없이 포함된 경우 해당 요청을 Block 하도록 규칙을 설정하도록 합니다.

- WAF Name: `wsc-waf`

14. CloudFront

CloudFront를 통하여 정적 콘텐츠 및 애플리케이션에 접근이 가능하도록 합니다. S3와 ALB를 Origin으로 사용하여 구성합니다. S3에 업로드되는 정적 콘텐츠를 캐싱할 수 있어야 하며, ALB로의 요청에 대해서는 캐싱하지 않고 Query String도 모두 Origin으로 전달해야 합니다. ALB는 VPC Origin을 사용하여 Internal ALB의 트래픽을 처리하여야 합니다. 사용자가 CloudFront에 HTTP로 접근 시 HTTPS로 리다이렉트 되도록 설정합니다. 채점 시 오동작을 방지하기 위해 IPv6는 비활성화하고, 하나의 CloudFront만 생성하도록 합니다.

- Origin : S3와 ALB Origin을 가지도록 구성
- Edge : 한국뿐만 아니라 전 세계의 유저가 빠른 접근할 수 있도록 구성
- CloudFront Name : wsc-cdn

15. Lambda

DynamoDB에 저장된 데이터를 조회하기 위한 API를 Lambda를 통해 구현합니다. 해당 Lambda는 Private Subnet 내에서 운용되어야 하며, wsc-app-lb의 Target으로 등록되어야 합니다. 또한 CloudFront를 통해 API 요청 시 정상적으로 접근이 가능하여야 합니다. API Spec은 아래를 참고하여 구성하며, 만약 DynamoDB Table에 데이터가 존재하지 않을 시 Status Code 404와 {"msg": "Item not found"}를 반환하도록 합니다. 자세한 건 아래를 참고하여 구성합니다.

- Lambda Function Name: wsc-get-table-function
- Runtime: Python 3.14
- API Spec

Path	Method	Description
/v1/book	GET	Request Body
		/v1/book?client_id=C001
		Response Body
		{ "username": "Alice", "booking_id": "6WVB5S9G", "email": "kim@example.com", "client_id": "C001", "concert_name": "Seoul2025" }

Reference01

- VPC 정보

Name	CIDR
wsc-vpc	10.0.0.0/16

- Subnet & Route Table 정보

Name	CIDR	Route Table	Description
wsc-public-a	10.0.0.0/24	wsc-public-rtb	Direct Access
wsc-public-c	10.0.1.0/24	wsc-public-rtb	Direct Access
wsc-private-a	10.0.2.0/24	wsc-private-a-rtb	Internal Access
wsc-private-c	10.0.3.0/24	wsc-private-c-rtb	Internal Access
wsc-workload-a	10.0.4.0/24	wsc-workload-a-rtb	No Internet
wsc-workload-c	10.0.5.0/24	wsc-workload-c-rtb	No Internet

Reference02

Go에서 개발된 binary book이 배포됩니다. 해당 파일은 x86 기반 시스템에서 빌드하고 동작을 확인하였습니다.

- 어플리케이션 실행 시 TCP/8080 포트로 바인딩 됩니다.
- 어플리케이션은 표준 출력으로 접근 로그를 출력합니다.
- 어플리케이션은 /health 경로로 상태 확인을 제공합니다.
- 어플리케이션은 환경변수를 통해 데이터베이스 연결 정보를 어플리케이션에 제공합니다. 환경변수 키 값은 어플리케이션의 설명을 참고합니다.

- API Spec

Path	Method	Description
/v1/book	POST	Request Body
		{ "client_id": "C001", "username": "Alice", "email": "kim@example.com", "concert_name": "Seoul2025" }
		Response Body
/health	GET	{"booking_id": "C2011YY"}
		{"status": "ok"}

- Environment Variables

Environment Key	Environment Value
AWS_REGION	ap-northeast-2
TABLE_NAME	wsc-table